Computación en Internet III
Ingeniería de Sistemas
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas

Examen NestJS: Computación en internet III
Docentes: Kevin Rodríguez
Tiempo para desarrollarlo: 2 horas
Enlace del classroom: https://classroom.github.com/a/zwx8q-wz

Restricciones y condiciones:

- El estudiante podrá visualizar sus **apuntes del curso,** así como la **documentación de curso** y **enlaces relacionados al contexto del ejercicio**. Podrá hacer uso de internet y búsquedas de código a través del navegador.
- El estudiante no podrá hacer uso de **ningún tipo de herramienta relacionada con IA Generativa** como lo son: Chat GPT, Gemini, Copilot, Claude y similares. De ser utilizado se anulará el examen y se hará el respectivo reporte al departamento respectivo para **iniciar un proceso disciplinario**.
- El estudiante no podrá hacer uso de **ningún dispositivo electrónico** además de su computador personal o el suministrado por el laboratorio para realizar la evaluación.
- El estudiante deberá hacer uso de las herramientas utilizadas dentro del curso hasta el momento, especialmente **Nest, TypeORM y Docker.**
- Dentro del examen, el docente le solicitará hacer commits a su repositorio de manera frecuente, esto con el objetivo de demostrar su desarrollo y avance a través del examen. Si su entrega final no posee un avance demostrable, su entrega será revisada posteriormente para demostrar su autenticidad.

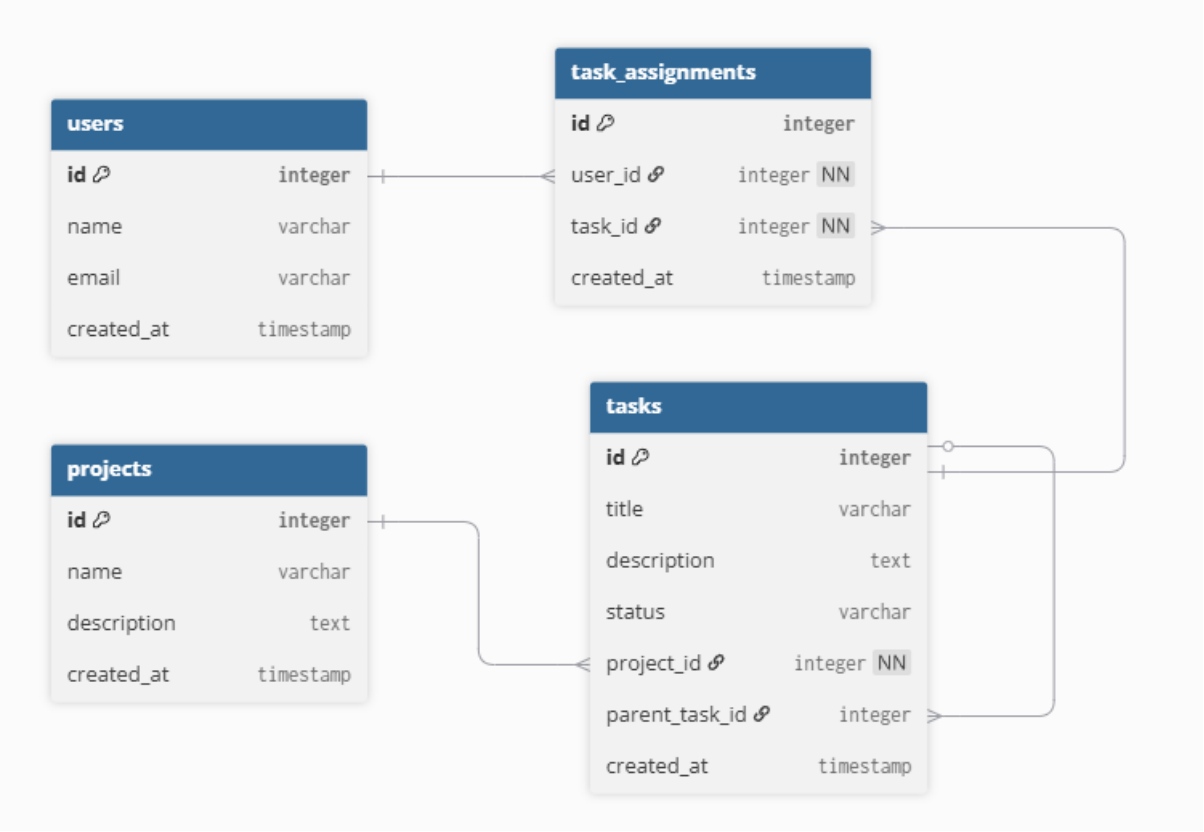
Nota final: Si la aplicación falla al ejecutarse, su nota podrá verse afectada drásticamente.

Objetivo de la asignación:

Una empresa de desarrollo de software está construyendo el backend de un sistema de gestión de proyectos basado en la metodología Kanban. En este sistema, los proyectos se componen de tareas que pueden tener subtareas asociadas, permitiendo descomponer el trabajo en unidades más pequeñas y manejables. **Cada tarea sigue un flujo de estados (*TODO, IN_PROGRESS, REVIEW, DONE*) y puede ser asignada a uno o más usuarios.**

Los usuarios pueden crear tareas, asignarse a ellas y actualizar su estado a medida que avanzan en el trabajo. Sin embargo, el sistema debe garantizar que las transiciones de estado y las acciones realizadas respeten ciertas reglas de negocio, especialmente en relación con la asignación de usuarios y la finalización de subtareas antes de avanzar en tareas principales.

Computación en Internet III
Ingeniería de Sistemas
Facultad Barberi de Ingeniería, Diseño y Ciencias Aplicadas



Para la primera iteración de este desarrollo, se ha solicitado que las entidades se definan utilizando el ORM TypeORM dentro de un backend desarrollado en NestJS.

No es necesario implementar la totalidad de las operaciones CRUD para todas las entidades. En su lugar, el desarrollador deberá implementar únicamente aquellas operaciones que considere necesarias para poder probar y validar correctamente las siguientes reglas de negocio:

1. Una tarea solo puede pasar al estado **IN_PROGRESS** si tiene al menos un usuario asignado.
    a. Si no tiene usuarios asignados, el sistema deberá rechazar el cambio de estado.
2. Una tarea padre no puede cambiar a estado **REVIEW** ni **DONE** si alguna de sus subtareas no se encuentra en estado **DONE**.
    a. El sistema debe validar el estado de todas las subtareas antes de permitir el cambio.
3. Una subtarea no puede ser creada sin estar asociada a una tarea padre válida.
    a. Si se intenta crear una subtarea sin una tarea padre existente, el sistema deberá rechazar la operación.
4. Una tarea que se encuentre en estado **DONE** no puede ser modificada ni cambiar nuevamente de estado.
5. El sistema debe bloquear cualquier intento de actualización sobre tareas finalizadas.

El esquema de pruebas de la aplicación debe ser realizado en Postman y deberá proveer el archivo JSON para verificar su funcionamiento.

Puntos de evaluación:

[10%]: Creación de módulos necesarios en el contexto de NestJS.
[30%]: Creación de las entidades especificadas en el contexto.
[30%]: Capa de servicio y controlador de las entidades especificadas.
[20%]: Integración de las reglas del negocio.
[10%]: Calidad de código, manejo de excepciones, validación de campos.

Sugerencias:

1. Puede especificar todas las entidades en un solo módulo.
2. Puede omitir la restricción a nivel de columnas que podrían tener cada entidad, a excepción de la no-nulidad.